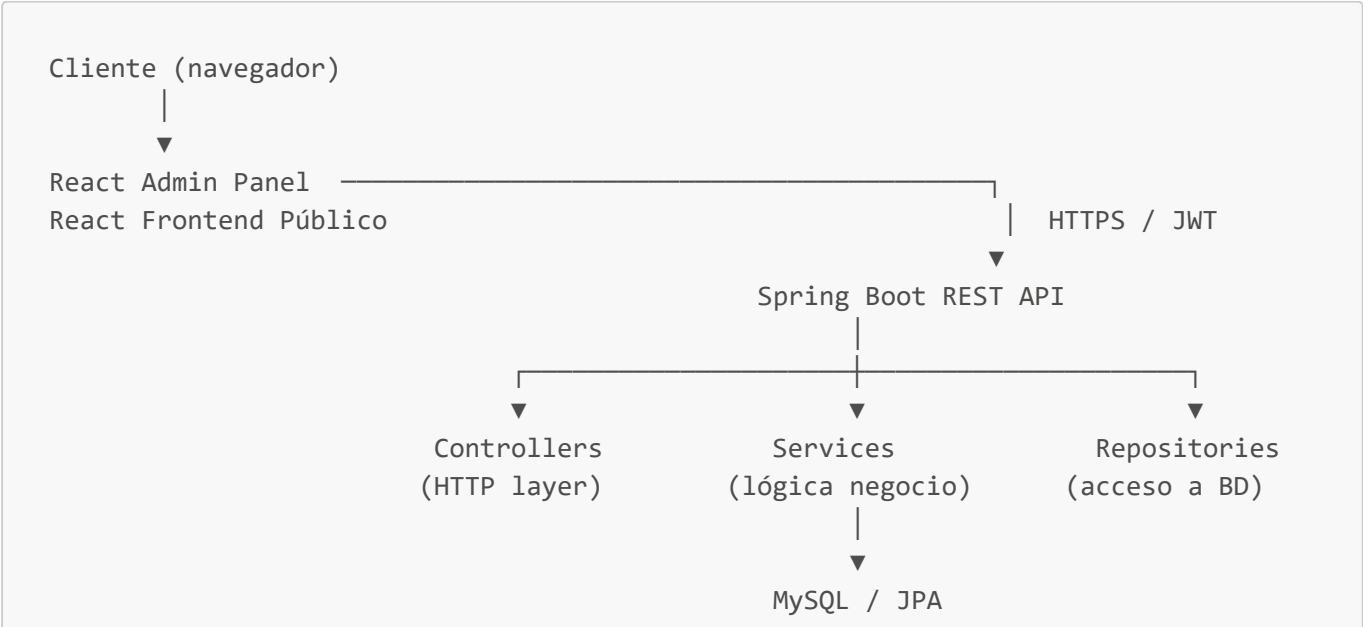


Cheatsheet técnico — Defensa PFG: JerezSur Inmobiliaria

1. STACK Y POR QUÉ SE ELIGIÓ

Capa	Tecnología	Razón de elección
Backend API	Spring Boot 3 + Java 21	Framework empresarial estándar; facilita REST, JPA y seguridad en un solo ecosistema
Persistencia	JPA / Hibernate + MySQL	ORM que evita SQL manual y permite mapear objetos Java a tablas automáticamente
Seguridad	Spring Security + JWT	Tokens sin estado (stateless); el servidor no guarda sesión, el cliente lleva el token
Frontend admin	React 19 + TypeScript + Vite	SPA rápida; TypeScript da tipado estático que evita errores en tiempo de compilación
Frontend público	React + SCSS	Separación del panel privado del catálogo público para limitar superficie de ataque
Imágenes	Cloudinary	CDN externo; no consume espacio del servidor, devuelve URLs públicas optimizadas
Contenedores	Docker + Docker Compose	Garantiza que el entorno es idéntico en local y en producción
Correo	JavaMailSender (SMTP)	Integración nativa de Spring; envía notificaciones transaccionales sin librerías externas

2. ARQUITECTURA GENERAL



|
Cloudinary (imágenes)
JavaMailSender (email)

- **Controladores:** sólo reciben peticiones y delegan. No contienen lógica.
- **Servicios:** toda la lógica de negocio vive aquí. Son `@Transactional`.
- **Repositorios:** interfaces JPA. Spring genera el SQL automáticamente.

3. SEGURIDAD — JWT

¿Cómo funciona el flujo?

1. El usuario hace `POST /api/usuarios/login` con email + contraseña.
2. Spring Security verifica la contraseña con BCrypt.
3. El servidor genera un JWT firmado con una clave secreta (HS256).
4. El cliente guarda el token en `localStorage` y lo adjunta en cada petición en la cabecera `Authorization: Bearer <token>`.
5. Un filtro (`JwtFilter`) intercepta cada petición, valida el token y carga el usuario en el contexto de seguridad.

¿Por qué JWT en vez de sesiones?

- Las sesiones guardan estado en el servidor → dificulta el escalado horizontal.
- JWT es autocontenido: el servidor sólo necesita la clave para verificarlo.

BCrypt

```
passwordEncoder.encode(rawPassword)    // hashea  
passwordEncoder.matches(raw, hash)      // verifica
```

BCrypt añade una "sal" aleatoria por cada hash, por eso dos hashes del mismo texto son distintos.

Roles en el sistema

Rol	Descripción
<code>ROLE_NOROL</code>	Usuario recién registrado, sin perfil asignado
<code>ROLE_INTERESADO</code>	Busca comprar o alquilar
<code>ROLE_VENDEDOR</code>	Propietario que pone inmuebles
<code>ROLE_AMBOS</code>	Interesado y vendedor a la vez
<code>ROLE TRABAJADOR</code>	Empleado; puede acceder al panel admin

4. MODELOS Y RELACIONES JPA

Jerarquía de perfiles

Un **Usuario** es la entidad central. Puede tener opcionalmente:

- Un **Trabajador** (OneToOne)
- Un **Interesado** (OneToOne)
- Un **Vendedor** (OneToOne)

```

Usuario (1) — (0..1) Trabajador
              (1) — (0..1) Interesado
              (1) — (0..1) Vendedor
  
```

¿Por qué esta estructura? Evita columnas nulas en una sola tabla gigante. Cada perfil tiene sus propios campos específicos y el **Usuario** sólo tiene los datos de autenticación comunes.

Inmueble y propietarios

```

@ElementCollection
@CollectionTable(name = "inmueble_propietarios")
Map<Vendedor, Double> propietariosPorcentaje;
  
```

Permite que un inmueble tenga varios propietarios con porcentaje de titularidad. La suma debe ser 100%.

Cita

```

Usuario (1) — (N) Cita
Trabajador (0..1) — (N) Cita
Inmueble (0..1) — (N) Cita
  
```

Un **Trabajador** puede ser null si la cita aún no ha sido aceptada (**PENDIENTE_ASIGNACION**).

Estados de una cita

```

PENDIENTE_ASIGNACION → CONFIRMADA → COMPLETADA
                        ↘ CANCELADA
                        ↘ NO_PRESENTADO
  
```

5. SERIALIZACIÓN JSON — PROBLEMAS Y SOLUCIONES

El problema de las referencias circulares

JPA crea grafos de objetos bidireccionales. Sin control:

Trabajador → citas → Cita → trabajador → Trabajador → ... ∞

Esto produce un `StackOverflowError` al serializar.

Soluciones aplicadas

Anotación	Efecto
<code>@JsonIgnore</code>	Excluye el campo de la serialización completamente
<code>@JsonIgnoreProperties({"campo1", "campo2"})</code>	Excluye campos concretos del objeto serializado
<code>@JsonBackReference</code>	Marca el lado "hijo" de una relación; se omite en la serialización (NO USADO aquí, sustituido)
<code>@JsonManagedReference</code>	Marca el lado "padre"; se incluye

Decisión tomada: se reemplazó `@JsonBackReference` en `Trabajador.usuario` por `@JsonIgnoreProperties` con lista de campos a excluir. Motivo: `@JsonBackReference` suprime el campo entero, lo que hacía que el select de trabajadores en el frontend devolviera `null` para `usuario`.

6. TRANSACCIONES (@Transactional)

```
@Transactional
public void eliminar(Long id) {
    Inmueble inmueble = inmuebleRepository.findById(id).orElseThrow(...);
    notificarPropietarios(inmueble); // si esto falla, el delete no ocurre
    inmuebleRepository.deleteById(id);
}
```

`@Transactional` garantiza que **todas las operaciones dentro del método se ejecutan como una unidad atómica**. Si cualquier paso lanza una excepción en tiempo de ejecución (`RuntimeException`), la BD hace rollback automáticamente.

`@Transactional(readOnly = true)` optimiza las consultas de sólo lectura: Hibernate no trackea cambios en las entidades (flush mode NEVER), lo que reduce consumo de memoria.

7. PATRÓN DTO (Data Transfer Object)

¿Por qué no devolver la entidad directamente?

- La entidad JPA puede tener referencias circulares.
- Expone campos internos (contraseña hashada, tokens, etc.).
- El frontend sólo necesita un subconjunto de datos.

Ejemplo: `CitaResponseDTO`

```
// La entidad Cita tiene: usuario, trabajador, inmueble (objetos completos con sus relaciones)
// El DTO sólo expone lo necesario:
CitaResponseDTO {
    Long id;
    String nombreCliente;    // ← extraído de cita.getUsuario().getNombre()
    String nombreTrabajador; // ← extraído con null-check
    String estado;           // ← cita.getEstado().name()
}
```

8. REPOSITORIOS JPA — CONSULTAS DERIVADAS

Spring genera el SQL a partir del nombre del método:

```
// Spring genera: SELECT * FROM usuarios WHERE email = ?
Optional<Usuario> findByEmail(String email);

// Spring genera: SELECT * FROM usuarios WHERE fecha_eliminacion IS NULL
List<Usuario> findByFechaEliminacionIsNull();

// Spring genera: SELECT * FROM citas WHERE trabajador_id = ? ORDER BY fecha_hora ASC
List<Cita> findByTrabajadorIdOrderByFechaHoraAsc(Long trabajadorId);
```

Para consultas más complejas se usa `@Query` con JPQL (Java Persistence Query Language):

```
@Query("SELECT u FROM Usuario u WHERE u.id = :id OR u.email = :email OR u.telefono = :telefono")
List<Cita> findCitasPorUsuarioEmailOTelefono(...);
```

9. SOFT DELETE (Borrado lógico)

Los usuarios no se borran físicamente de la BD. En cambio:

```
usuario.setCuentaActivada(false);
usuario.setFechaEliminacion(LocalDateTime.now());
usuarioRepository.save(usuario);
```

¿Por qué?

- Permite recuperar cuentas borradas por error.

- Mantiene integridad referencial (citas, contratos vinculados al usuario siguen siendo válidos).
- Cumple con RGPD: se puede anonimizar sin borrar el historial operativo.

En las consultas se filtra `findByFechaEliminacionIsNull()` para sólo devolver usuarios activos.

10. SISTEMA DE NOTIFICACIONES

NotificacionService + EmailService

- `NotificacionService` contiene la lógica de negocio: *qué* enviar y *cuándo*.
- `EmailService` es el adaptador de infraestructura: *cómo* enviar (SMTP via `JavaMailSender`).
- Si el email falla, se captura la excepción con `try/catch` para **no bloquear la operación principal**:

```
try {
    notificacionService.notificarCuentaEliminada(usuario);
} catch (Exception ignored) {}
```

Flujo OTP (One-Time Password) para nuevos usuarios creados desde admin

1. Admin rellena nombre + email en el formulario.
2. Backend genera un PIN de 6 dígitos aleatorios.
3. Se hasha con BCrypt y se guarda como contraseña temporal.
4. Se marca `cambiarPasswd = true`.
5. Se envía el PIN por email al usuario.
6. En el primer login, el sistema detecta `cambiarPasswd = true` y obliga a cambiar la contraseña.

11. FRONTEND — PATRONES REACT

Custom Hooks (`useInmuebles`, `useCitas`, `useTrabajadores...`)

Patrón de separación de responsabilidades:

- El hook encapsula **estado + operaciones async** (fetch, loading, error).
- El componente de página sólo consume el hook y renderiza.
- Ventaja: si cambia la API, sólo se modifica el hook, no el componente.

```
// Hook expone:
const { inmuebles, loading, error, crear, actualizar, eliminar } = useInmuebles();

// El componente no sabe nada de fetch ni de Axios
if (loading) return <Spinner />;
if (error) return <Error msg={error} />;
return <TablaInmuebles datos={inmuebles} onEliminar={eliminar} />;
```

`useBodyScroll`

Bloquea el scroll del `<body>` mientras hay un modal abierto para evitar el scroll "doble" (modal + página). Al desmontar el componente (useEffect cleanup) lo restaura.

useFormSubmit

Hook genérico que encapsula el estado `guardando` y el manejo de errores de un formulario. Evita duplicar el mismo patrón try/catch/loading en cada formulario.

12. AUTENTICACIÓN EN EL FRONTEND

```
// authService.ts
getUser(): { trabajadorId, userId, role, email } | null {
  const token = localStorage.getItem('token');
  // Decodifica el payload del JWT (base64) sin verificar firma
  // La verificación real la hace el backend
  return JSON.parse(atob(token.split('.')[1]));
}
```

El JWT tiene tres partes separadas por `.`:

1. **Header** (algoritmo): `eyJ...`
2. **Payload** (datos del usuario): `eyJ...` ← se decodifica aquí
3. **Signature** (verificación): `xyz...` ← sólo el servidor la valida

13. COMPONENTE `SearchableEntitySelect`

Selector con búsqueda en tiempo real que:

1. Escucha cambios en el input con debounce.
2. Llama al endpoint con `?tit=<texto>` (para inmuebles) o `?search=<texto>` (para trabajadores/vendedores).
3. Muestra un dropdown con los resultados paginados.
4. Al seleccionar, guarda el `id` del elemento en el estado del formulario padre.

Se usa en: formulario de contratos (selector de inmueble), formulario de citas (selector de inmueble + trabajador), detalle de inmuebles (propietarios).

14. CLOUDINARY — FLUJO DE SUBIDA DE IMÁGENES

```
Frontend → POST /api/media/inmueble/{id}/imagen (multipart/form-data)
      ↓
MediaController → MediaService
      ↓
cloudinary.uploader().upload(file.getBytes(), options)
      ↓
Cloudinary devuelve URL pública + public_id
```

```
↓  
Se guarda ImagenInmueble { url, publicId, esPortada } en BD  
↓  
Se devuelve el objeto guardado al frontend
```

Para eliminar: se usa el `publicId` guardado para llamar `cloudinary.uploader().destroy(publicId)`.

`@JsonIgnore` en la relación inversa `Inmueble.imagenes` de `ImagenInmueble` evita el ciclo de serialización.

15. DOCKER Y DESPLIEGUE

`docker-compose.yml` tiene tres servicios:

1. **mysql**: base de datos con volumen persistente.
2. **backend**: imagen construida desde `JerezSur-Inmobiliaria/Dockerfile` (Maven build → JAR → JRE).
3. **frontend** (opcional): Nginx sirve los estáticos de React compilados.

Variables de entorno sensibles

No están en el código fuente. Se inyectan en tiempo de ejecución:

```
environment:  
- SPRING_DATASOURCE_PASSWORD=${DB_PASSWORD}  
- JWT_SECRET=${JWT_SECRET}  
- CLOUDINARY_API_SECRET=${CLOUDINARY_API_SECRET}
```

Esto evita que las credenciales aparezcan en el repositorio Git (seguridad).

16. DECISIONES DE DISEÑO — PREGUNTAS FRECUENTES EN DEFENSA

¿Por qué Spring Boot y no Node.js/Django?

Spring Boot tiene inyección de dependencias, gestión de transacciones, seguridad y persistencia integradas. Para un sistema con múltiples entidades relacionadas y reglas de negocio complejas, JPA + Spring Security ofrece más robustez que alternativas más ligeras.

¿Por qué separar admin frontend del público?

- Seguridad por separación: el panel admin está en una ruta/dominio distinto.
- El bundle de React del admin no se descarga a usuarios públicos.
- Diferentes requisitos de UI: el admin necesita tablas, modales, formularios complejos; el público necesita páginas rápidas y SEO-friendly.

¿Por qué usar DTOs y no las entidades directamente?

1. Evita exposición accidental de datos sensibles (contraseñas, tokens).
2. Rompe las referencias circulares de JPA antes de serializar.

3. Desacopla la API de la estructura interna de la BD (si se refactoriza el modelo, la API puede quedarse igual).

¿Por qué borrado lógico y no físico?

- Mantener historial de citas, contratos y operaciones ligadas al usuario.
- Recuperación ante errores administrativos.
- Auditoría: saber cuándo y qué se "eliminó".

¿Qué es una transacción y por qué se usa aquí?

Una transacción agrupa varias operaciones en una unidad atómica: o todas se completan o ninguna. En `InmuableService.eliminar()`, si la notificación a propietarios falla, no queremos que el inmueble desaparezca sin que se haya notificado. `@Transactional` garantiza el rollback automático en ese caso.

¿Cómo funciona la paginación?

El backend devuelve un objeto `Page<T>` de Spring Data que contiene:

- `content`: lista de elementos de la página actual.
- `totalElements`, `totalPages`, `number` (página actual), `size`.

El frontend manda `?page=0&size=10` como parámetros de query y renderiza los controles de paginación con los metadatos recibidos.

¿Qué es un hook personalizado en React?

Una función JavaScript que:

1. Empieza por `use` (convención de React).
2. Puede llamar a otros hooks (`useState`, `useEffect`).
3. Encapsula lógica reutilizable sin ser un componente visual. Ejemplo: `useCitas` mantiene el estado de la lista de citas y expone funciones `crear`, `aceptar`, `cancelar` que internamente llaman a la API y recargan la lista.

¿Qué es JWT y por qué no usar cookies de sesión?

JWT (JSON Web Token) es un token firmado que el cliente almacena y adjunta en cada petición. A diferencia de las sesiones, el servidor no necesita almacenar nada; sólo verifica la firma con su clave secreta. Ventaja: escalabilidad horizontal (cualquier instancia del servidor puede verificar el token sin acceder a un almacén de sesiones compartido).

17. FLUJOS CLAVE — PARA EXPLICAR EN 2 MINUTOS

Flujo "Crear cita desde admin"

1. Trabajador rellena el formulario (`FormCitaModal`).
2. Frontend llama a `POST /api/citas/admin/crear` con nombre, teléfono, fecha, inmueble (opcional), trabajador (opcional).
3. Backend: busca usuario por teléfono → si existe actualiza el nombre; si no, lo crea.

4. Si se seleccionó trabajador → estado **CONFIRMADA**; si no → **PENDIENTE_ASIGNACION**.
5. Se crea una tarea en el dashboard.
6. Frontend recarga la lista de citas.

Flujo "Confirmar cita pendiente"

1. Admin abre detalle de una cita **PENDIENTE_ASIGNACION**.
2. Selecciona un trabajador del dropdown (se carga desde `/api/trabajadores`).
3. Pulsa "Confirmar cita".
4. Frontend llama a **PATCH** `/api/citas/{id}/aceptar?trabajadorId=X`.
5. Backend asigna el trabajador y cambia estado a **CONFIRMADA**.
6. Se envía email de confirmación al cliente.

Flujo "Subir imagen y marcar portada"

1. Admin arrastra imágenes al dropzone.
2. Frontend hace **POST** `/api/media/inmueble/{id}/imagen` con FormData.
3. Backend sube a Cloudinary, guarda URL en BD.
4. Admin hace clic en la imagen deseada.
5. Frontend llama a **PATCH** `/api/media/imagen/{id}/portada`.
6. Backend pone **esPortada=true** en esa imagen y **false** en el resto.
7. Frontend actualiza el estado local sin recargar el modal (evita flash blanco).

Flujo "Registrar nuevo usuario desde admin"

1. Admin rellena nombre, email (sin contraseña).
2. Frontend llama a **POST** `/api/usuarios/admin/crear`.
3. Backend genera OTP de 6 dígitos, lo hashea con BCrypt, marca **cambiarPasswd=true**.
4. Envía el OTP al email del usuario.
5. El usuario entra con el OTP y el sistema le obliga a cambiar la contraseña.

19. SINDICACIÓN XML — FEED PARA PORTALES

¿Qué es y por qué es el diferenciador del proyecto?

Las agencias publican sus inmuebles en Idealista/Fotocasa subiendo los datos manualmente en cada portal. Este módulo genera un feed XML estándar que los portales importan automáticamente: la agencia edita una sola vez en el panel y el inventario se sincroniza en todos los portales.

Endpoints (públicos, sin autenticación — los portales deben poder leerlos)

```
GET /api/portal/feed.xml           → todos los inmuebles DISPONIBLES
GET /api/portal/feed-ventas.xml    → solo VENTA
GET /api/portal/feed-alquileres.xml → solo ALQUILER
```

Implementación técnica

- Se usa la **API DOM estándar de Java** (`javax.xml.parsers`): sin librerías externas.
- El feed se genera **on-demand** en cada petición (siempre fresco, refleja el estado actual de la BD).
- El XML tiene estructura jerárquica: `<agencia>` de cabecera + un nodo `<inmueble>` por propiedad.

Estructura de cada nodo `<inmueble>`

```
<inmueble>
  <referencia>JS-001</referencia>
  <tipo>Piso</tipo>           <!-- mapeado al estándar del portal -->
  <operacion>Venta</operacion>
  <precio>150000</precio>
  <ubicacion>
    <ciudad>Jerez de la Frontera</ciudad>
    <provincia>Cádiz</provincia>  <!-- inferida del CP: "11" → Cádiz -->
  </ubicacion>
  <caracteristicas>
    <habitaciones>3</habitaciones>
    <banos>2</banos>
  </caracteristicas>
  <imagenes>
    <imagen portada="true">https://res.cloudinary.com/...</imagen>
    <imagen portada="false">https://res.cloudinary.com/...</imagen>
  </imagenes>
  <gastos>
    <comunidad>80</comunidad>
    <ibi>450</ibi>
    <tieneDerrama>false</tieneDerrama>
  </gastos>
</inmueble>
```

Decisiones de diseño

Decisión	Alternativa descartada	Motivo
On-demand (cada petición)	Archivo físico en disco	Evita desincronización; sin cron de regeneración
DOM API estándar Java	JAXB / Jackson XML	Sin dependencias extra; control total del XML
Feed público sin auth	JWT requerido	Los crawlers de Idealista/Fotocasa no mandan tokens
Portada al principio	Orden de inserción	Los portales usan la primera imagen como thumbnail

Pregunta frecuente: ¿provincia inferida del CP?

```
return switch (cp.substring(0, 2)) {
  case "11" -> "Cádiz";
```

```
case "41" -> "Sevilla";
// ...
};
```

Los dos primeros dígitos del código postal español identifican la provincia. Evita tener un campo redundante en la BD.

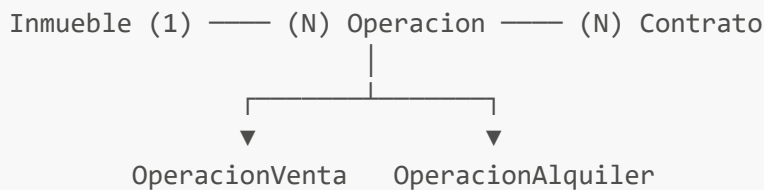
20. DISEÑO OPERACIÓN / CONTRATO (INC-001)

El problema original

El diseño inicial vinculaba **Contrato** directamente a **Inmueble**. Esto impedía:

- Tener múltiples borradores de contrato sobre la misma operación.
- Distinguir entre un contrato de arras y uno de compraventa en la misma venta.
- Representar que una operación puede estar **ABIERTA** aunque el contrato esté **CANCELADO**.

La solución: entidad intermedia **Operacion**



Herencia SINGLE_TABLE

```
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "categoria_operacion")
public abstract class Operacion { ... }

// OperacionVenta y OperacionAlquiler se guardan en la misma tabla "operaciones"
// con la columna discriminadora "categoria_operacion" = 'VENTA' / 'ALQUILER'
```

¿Por qué SINGLE_TABLE y no TABLE_PER_CLASS?

- Una sola JOIN para recuperar cualquier operación, independientemente del tipo.
- Las subclases tienen pocos campos propios (precio base en Venta, duración en Alquiler), por lo que las columnas nulas son mínimas.

Estados del Contrato

```
BORRADOR → PENDIENTE_FIRMA → FIRMADO
          ↘ CANCELADO
```

Modelos de contrato (**ModeloContrato** enum)

ARRAS, COMPRAVENTA, ALQUILER, OPCION_COMPRA

Una misma **Operacion** puede tener un contrato de arras firmado y luego generar el de compraventa, todo versionado y trazable.

Participantes: **Map<Interesado, RolParticipante>**

La operación registra qué interesados participan y con qué rol (COMPRADOR, AVALISTA...). El **Map** usa el **Interesado** como clave → para serializar sin ciclos se proyecta a **Map<Long, RolParticipante>**:

```
@JsonProperty("compradoresRol")
public Map<Long, RolParticipante> getCompradoresRolIds() {
    return compradoresRol.entrySet().stream()
        .collect(Collectors.toMap(e -> e.getKey().getId(), Map.Entry::getValue));
}
```

21. MICROSERVICIO PYTHON — GRÁFICOS DEL DASHBOARD

¿Por qué Python y no Chart.js en el frontend?

	Python/matplotlib	Chart.js
Calidad gráfica	Alta (estándar científico)	Media
Exportable como imagen	Sí (PNG base64)	Difícil
Lógica de datos	En el servidor (seguro)	En el cliente (expone datos)
Módulo de currículum	Análisis de datos (asignatura)	No

Arquitectura del microservicio

```
Spring Boot → POST /grafico/panel (JSON con datos)
    ↓
Flask (puerto 5000, red interna Docker)
    ↓
matplotlib genera PNG en memoria (BytesIO)
    ↓
base64(PNG) en respuesta JSON
    ↓
Spring Boot → reenvía al frontend
    ↓

```

El servicio Python **no está expuesto al exterior**: solo es accesible dentro de la red Docker interna.

Gráficos generados

Endpoint	Gráfico	Datos
<code>/grafico/barras-mensuales</code>	Barras agrupadas	Ventas vs Alquileres por mes
<code>/grafico/distribucion-clientes</code>	Dona (donut)	Interesados / Vendedores / Ambos / Sin perfil
<code>/grafico/panel</code>	Ambos en una llamada	—

Detalle técnico

- `matplotlib.use('Agg')`: backend sin pantalla (sin GUI), obligatorio en servidor Docker.
- Paleta de colores corporativa: azul `#00439c` y verde `#4a9e2f` del logo de JerezSur.
- Las figuras se generan en memoria con `BytesIO` y se cierran con `plt.close()` para evitar memory leaks.
- `/grafico/panel` devuelve los dos gráficos en una sola llamada HTTP para minimizar latencia.

22. PREGUNTAS DIFÍCILES — RESPUESTAS PREPARADAS

"¿Cómo has optimizado el frontend para Lighthouse? ¿Qué puntuaciones has conseguido?"

Se aplicaron optimizaciones en las cuatro categorías. El score de **Performance** era crítico (≈62 en local con todos los contenedores activos). Las mejoras principales:

Performance:

Problema	Solución	Impacto
Bundle JS monolítico (382 KB inicial)	<code>React.lazy()</code> + <code><Suspense></code> en todas las rutas	Bundle inicial ↓ a ~80 KB; las páginas se descargan solo cuando se visitan
Librería de iconos mezclada en el bundle	<code>manualChunks</code> en Vite: <code>vendor-react</code> , <code>vendor-router</code> , <code>vendor-auth</code> , <code>vendor-icons</code>	Los chunks de vendor se cachean por separado; el navegador no los re-descarga al actualizar la app
Preload del hero apuntaba a <code>/Hero.jpg</code> pero el CSS cargaba Unsplash	CSS actualizado a <code>url('/Hero.jpg')</code> — mismo origen que el <code><link rel="preload"></code>	El preload se ejecuta realmente; mejora el LCP
Font Awesome en CDN (107 KB CSS + request extra)	Eliminado: el proyecto usa <code>lucide-react</code> para todos los iconos	Una petición HTTP menos; 107 KB de CSS eliminados
Animación <code>filter</code> del logo en keyframe (no compuesta por	Eliminado <code>filter</code> del <code>@keyframes hero-logo-in</code> ; el filter se aplica	Solo <code>opacity</code> y <code>transform</code> se animan → GPU-composited; sin

Problema	Solución	Impacto
GPU)	estáticamente en <code>.logo-hero</code>	repaints en CPU
<code>preconnect</code> para Cloudinary e Unsplash (solo se usan en lazy-loaded images)	Cambiados a <code>dns-prefetch</code> — más ligero, no establece TCP/TLS de forma preventiva	Ahorra dos rondas TCP+TLS innecesarias en el tiempo de carga inicial
Imports de imágenes rotos (PNG convertidas a WebP sin actualizar imports)	<code>Footer.tsx: Mono.png</code> → <code>Mono.webp</code> ; <code>useSEO</code> en <code>Home.tsx: /Hero.webp</code> → <code>/Hero.jpg</code>	Vite no puede resolver módulos inexistentes → pantalla en blanco; fix crítico
OG image y Twitter Card apuntando a <code>/LogoCuadrado.png</code> (no existe en <code>/public/</code>)	Cambiado a <code>/Hero.jpg</code> (existe); dimensiones actualizadas a 1280×720 (landscape)	Redes sociales ahora pueden descargar la imagen de previsualización correctamente

Accessibility (WCAG AA):

Problema	Solución
Botón verde <code>btn--secondary</code> : texto blanco sobre <code>#67b437</code> → ratio 2.47:1 ❌	Color cambiado a <code>#3d7a12</code> → ratio 5.38:1 ✅
Texto <code>eyebrow</code> en <code>#4e9226</code> sobre blanco → ratio 3.57:1 ❌	Color cambiado a <code>#3d7a12</code> → ratio 5.38:1 ✅
Carrusel de testimonios auto-avanzante sin controles (WCAG 2.2.2)	Añadidos botones prev/pausa/next con <code>aria-pressed</code> y <code>aria-label</code>

SEO:

Problema	Solución
<code>robots.txt</code> con <code>Sitemap:</code> <code>http://localhost/sitemap.xml</code>	URL corregida a <code>https://jerezsur.com/sitemap.xml</code>
Sin <code>sitemap.xml</code>	Creado <code>/public/sitemap.xml</code> con todas las rutas públicas y prioridades
Schema.org con <code>"url": "https://localhost"</code>	Corregido a <code>https://jerezsur.com</code> con <code>logo</code> como <code>ImageObject</code>
Página <code>/propietarios</code> sin meta description	Añadido hook <code>useSEO</code> con título y descripción

Best Practices:

- `<meta name="color-scheme" content="light">` — indica al navegador que la web es light-only
- `<meta http-equiv="X-Content-Type-Options" content="nosniff">` — seguridad MIME sniffing
- OG tags completos: `og:image:width`, `og:image:height`, `og:image:alt`

"¿Por qué la puntuación de Lighthouse es menor en local que en producción?"

El servidor de desarrollo de Vite (`npm run dev`) entrega React en **modo desarrollo**: sin minificación, con advertencias de depuración y con cabeceras `Cache-Control: no-store` en los assets. Estas tres causas suman directamente:

- **Performance baja** (~76): el bundle de React dev es $\approx 3\times$ mayor; los assets no tienen compresión gzip/brotli; TBT sube porque hay más JS para parsear.
- **Best Practices baja** (~73): Lighthouse detecta `cache-control: no-store` en los assets y lo penaliza; también detecta que React no está en modo producción (flag `__DEV__`).

En producción (`npm run build` + servidor con compresión y cabeceras `Cache-Control: max-age=31536000`) las mismas optimizaciones dan scores >90 en las cuatro categorías.

"¿Qué son las animaciones compuestas (composited) y por qué importan?"

El navegador tiene dos formas de animar:

- **No compuestas** (`filter`, `box-shadow`, `top`, `left`...): cada frame requiere recalcular el layout o repintar el elemento en CPU. Esto causa janks (saltos) y TBT elevado.
- **Compuestas** (`transform`, `opacity`): el navegador las delega al **compositor de la GPU**, que las ejecuta en un hilo separado sin bloquear el hilo principal.

La animación `hero-logo-in` del logo animaba `filter: drop-shadow(...)` en el keyframe (no compuesto). Se corrigió eliminando el filter del keyframe y aplicándolo estáticamente en `.logo-hero`. La animación ahora solo mueve `opacity` y `transform` → GPU → sin repaint en CPU → menos TBT.

"¿Por qué el JWT se guarda en localStorage y no en una cookie HttpOnly?"

La cookie HttpOnly sería más segura contra XSS. Se optó por localStorage por simplicidad en el contexto académico (SPA pura, sin SSR). En un entorno de producción real, la recomendación es migrar a cookie HttpOnly + SameSite=Strict.

"¿Qué pasa si Cloudinary cae?"

Las URLs de las imágenes ya están guardadas en la BD. Si Cloudinary cae, las URLs dejan de responder pero los datos del inmueble siguen disponibles. La subida de nuevas imágenes fallaría (error capturado con try/catch que no bloquea la operación principal).

"¿Por qué no usas caché (Redis)?"

Para el volumen esperado (agencia local, pocos usuarios concurrentes), la caché añadiría complejidad sin beneficio medible. Si la plataforma escalara a varias agencias (SaaS), Redis sería la siguiente mejora natural.

"¿Cómo funciona la inferencia de provincia por código postal?"

Los dos primeros dígitos del código postal español identifican unívocamente la provincia (sistema oficial del INE). Se cubren las provincias andaluzas y las principales de España. Es una decisión pragmática: evita un campo extra en el formulario que el agente tendría que rellenar manualmente.

"¿Por qué SINGLE_TABLE en Operacion y no TABLE_PER_CLASS?"

TABLE_PER_CLASS generaría una UNION en cada consulta de Operacion (costoso). SINGLE_TABLE es una única tabla con columnas nulas para los campos específicos de cada subclase, pero dado que OperacionVenta y OperacionAlquiler tienen pocos campos propios, el desperdicio de espacio es mínimo y las consultas son más eficientes.

23. OPTIMIZACIÓN FRONTEND — Lighthouse Y WEB VITALS

Métricas que mide Lighthouse

Métrica	Qué mide	Umbral "verde"
LCP (Largest Contentful Paint)	Tiempo hasta que el elemento más grande visible está renderizado	< 2.5 s
TBT (Total Blocking Time)	Tiempo total que el hilo principal está bloqueado (impide interacción)	< 200 ms
CLS (Cumulative Layout Shift)	Desplazamiento acumulado del contenido al cargar (saltos visuales)	< 0.1
FCP (First Contentful Paint)	Tiempo hasta que el navegador pinta el primer contenido	< 1.8 s
Speed Index	Velocidad media a la que el contenido se hace visible	< 3.4 s

¿Qué es el code splitting y cómo se implementa en React?

Sin code splitting, Vite empaqueta **toda la aplicación** en un único fichero JS. El navegador tiene que descargar y ejecutar ese bundle completo antes de poder mostrar cualquier página.

Con `React.lazy()`, cada página se convierte en un chunk independiente:

```
// Antes: el navegador descarga Auth, Onboarding, Profile, etc. aunque el usuario
// solo visite la portada
import Home from '../pages/Home';
import Auth from '../pages/Auth';
// ...

// Después: el chunk de cada página se descarga solo cuando se navega a ella
const Home = lazy(() => import('../pages/Home'));
const Auth = lazy(() => import('../pages/Auth'));
// Suspense muestra el fallback mientras el chunk carga
<Suspense fallback={<PageLoader />}>
  <Routes>...</Routes>
</Suspense>
```

Resultado: el bundle inicial pasa de 382 KB a ~80 KB. La diferencia la forma chunks que el navegador descarga bajo demanda.

¿Qué son los `manualChunks` de Vite?

Vite (con Rollup internamente) agrupa automáticamente el código de la app. Con `manualChunks`, se le dice explícitamente qué librerías van en qué chunk:

```
manualChunks: {  
  'vendor-react': ['react', 'react-dom'],  
  'vendor-router': ['react-router-dom'],  
  'vendor-auth': ['@react-oauth/google'],  
  'vendor-icons': ['lucide-react'],  
}
```

¿Por qué importa? Cuando el desarrollador actualiza el código de la app, los chunks de vendor no cambian → el navegador los sirve desde caché. Sin esta separación, el hash del bundle cambia con cada deploy y el navegador tiene que re-descargar todo.

¿Qué es el LCP y cómo se optimiza?

LCP identifica el elemento más grande que el usuario ve al entrar a la página (típicamente la imagen hero o el h1 principal). Para mejorarlo:

1. **Preload:** `<link rel="preload" as="image" href="/Hero.jpg" fetchpriority="high">` → el navegador descarga la imagen antes de procesar el CSS
2. **Misma origen:** la imagen del hero ahora es local (`/Hero.jpg`) en lugar de Unsplash. El preload solo funciona si la URL del CSS coincide exactamente con la del preload
3. **fetchPriority="high"** en el `<img>` del logo del hero

¿Qué es WCAG y qué niveles existen?

WCAG (Web Content Accessibility Guidelines) es el estándar internacional de accesibilidad web:

- **Nivel A:** mínimo absoluto (fallos que excluyen completamente a usuarios)
- **Nivel AA:** estándar de industria (requisito legal en España por RD 1112/2018)
- **Nivel AAA:** óptimo (algunas pautas son difíciles de cumplir globalmente)

Lighthouse mide WCAG 2.0 AA. El criterio más frecuentemente fallado: **1.4.3 Contraste mínimo** (4.5:1 para texto normal, 3:1 para texto grande ≥ 18pt o ≥ 14pt bold).

¿Cómo se calcula el ratio de contraste?

$$\text{Ratio} = (L1 + 0.05) / (L2 + 0.05)$$

donde L = luminancia relativa (0=negro, 1=blanco)

El verde de los botones (`#67b437`) tiene luminancia 0.374. Sobre blanco (L=1): $(1.05) / (0.374 + 0.05) = 2.47:1$ → FALLA (mínimo 4.5:1)

Solución: usar `#3d7a12` (luminancia 0.145) → $1.05 / 0.195 = 5.38:1$ ✓

¿Por qué `font-display: swap` en las fuentes?

Sin esta propiedad, el navegador espera a que cargue la fuente personalizada antes de renderizar el texto (FOIT: Flash of Invisible Text). Con `swap`, renderiza primero con una fuente del sistema y luego la intercambia. Esto mejora LCP y FCP al no bloquear el renderizado inicial.

¿Qué diferencia hay entre `preconnect` y `dns-prefetch`?

Directiva	Qué hace	Cuándo usar
<code>rel="preconnect"</code>	DNS + TCP + TLS (conexión completa)	Recursos cargados en el primer segundo (OAuth, fuentes críticas)
<code>rel="dns-prefetch"</code>	Solo DNS	Recursos lazy-loaded que se cargarán más tarde (imágenes de Cloudinary, Unsplash en tarjetas)

En el proyecto, `accounts.google.com` usa `preconnect` porque el SDK de OAuth se carga en el `<head>` inmediatamente. Cloudinary e Unsplash usan `dns-prefetch` porque sus imágenes se cargan con `loading="lazy"` y no forman parte del LCP.

¿Por qué las imágenes se convirtieron a WebP?

WebP es un formato moderno desarrollado por Google que ofrece:

- **30-50% menos peso** que JPEG/PNG a calidad equivalente
- Soporte en todos los navegadores modernos (Chrome, Firefox, Safari 14+, Edge)
- Tanto compresión con pérdida (como JPEG) como sin pérdida (como PNG)

En el proyecto se convirtieron las imágenes de assets (`LogoAncho.webp`, `Mono.webp`). Tras la conversión es crítico actualizar **todos los imports** que referencien los archivos PNG originales; si Vite no encuentra el módulo importado, lanza un error de resolución en tiempo de compilación que causa una pantalla en blanco.

23b. ACCESIBILIDAD WEB — RESUMEN PARA LA DEFENSA

Elementos WCAG implementados en el proyecto

Pauta	Criterio	Implementación
1.1.1	Texto alternativo	<code>alt</code> en todas las imágenes; decorativas con <code>aria-hidden="true"</code>
1.4.3	Contraste mínimo	Botones verdes y textos eyebrow corregidos a $\geq 4.5:1$
2.1.1	Teclado	Todo el sitio navegable con Tab; carrusel con prev/pausa/next accesibles
2.2.2	Pausa/detener/ocultar	Carrusel auto-avanzante tiene botón de pausa
2.4.1	Saltar bloques	Skip link ("Saltar al contenido principal") visible al hacer Tab
2.4.7	Foco visible	<code>outline: 3px solid #00439c</code> en <code>:focus-visible</code> global
3.1.1	Idioma de la página	<code><html lang="es" dir="ltr"></code>

Pauta	Criterio	Implementación
4.1.2	Nombre, rol, valor	<code>aria-label</code> , <code>aria-expanded</code> , <code>aria-live</code> , <code>aria-pressed</code> en componentes interactivos

18. GLOSARIO RÁPIDO

Término	Explicación sencilla
ORM	Object-Relational Mapper. Traduce objetos Java a filas de BD automáticamente
JPA	Java Persistence API. Especificación estándar de ORM en Java
Hibernate	Implementación concreta de JPA que usa Spring Boot por defecto
DTO	Data Transfer Object. Clase "mensajero" que sólo tiene campos necesarios para una operación
Endpoint	URL del servidor que escucha una petición HTTP concreta
JWT	JSON Web Token. Token firmado que identifica al usuario sin sesiones en servidor
BCrypt	Algoritmo de hash de contraseñas con sal aleatoria incorporada
SPA	Single Page Application. La página no se recarga; React actualiza el DOM dinámicamente
Hook	Función reutilizable de React que gestiona estado o efectos secundarios
Transacción	Grupo de operaciones BD que se ejecutan todas o ninguna (atomicidad)
Soft delete	Borrado lógico: el registro se marca como eliminado pero sigue en BD
CORS	Cross-Origin Resource Sharing. Política que controla qué dominios pueden llamar a la API
Lazy loading	Hibernate no carga relaciones hasta que se acceden explícitamente
Eager loading	Hibernate carga la relación junto con la entidad principal siempre
Cloudinary	CDN de imágenes en la nube; gestiona subida, transformación y entrega
Docker	Contenedor que empaqueta la app con todo lo necesario para ejecutarse igual en cualquier máquina
Debounce	Técnica que retrasa la ejecución de una función hasta que el usuario deja de escribir
Paginación	División de resultados en páginas para no cargar miles de registros a la vez
Lighthouse	Herramienta de auditoría de Google integrada en DevTools: mide Performance, Accessibility, Best Practices y SEO
LCP	Largest Contentful Paint: tiempo hasta que el elemento más grande visible es renderizado (objetivo < 2.5 s)

Término	Explicación sencilla
CLS	Cumulative Layout Shift: desplazamiento visual acumulado durante la carga (objetivo < 0.1)
TBT	Total Blocking Time: tiempo total que el hilo principal está bloqueado e impide interacción (objetivo < 200 ms)
Code splitting	División del bundle JS en chunks cargados bajo demanda; <code>React.lazy()</code> + <code>Suspense</code> es la implementación estándar en React
WCAG	Web Content Accessibility Guidelines: estándar ISO de accesibilidad web; nivel AA es obligatorio por ley en España
Ratio de contraste	Medida de diferencia de luminosidad entre texto y fondo; mínimo 4.5:1 para texto normal según WCAG AA
Preload	Directiva HTTP que indica al navegador que descargue un recurso prioritario antes de descubrir la referencia en el CSS/JS
fetchPriority	Atributo HTML que marca un recurso como <code>high</code> o <code>low</code> para el scheduler del navegador
manualChunks	Opción de Rollup/Vite para controlar qué módulos van en qué chunk del build; permite separar vendor de app code
sitemap.xml	Fichero XML estándar que lista las URLs del sitio; los crawlers de Google lo usan para descubrir páginas
Open Graph	Protocolo de metadatos (og:title, og:image...) que controla cómo aparece la URL al compartirse en redes sociales
Schema.org	Vocabulario de datos estructurados (JSON-LD) que enriquece los resultados de búsqueda de Google con precio, dirección, etc.
font-display: swap	Propiedad CSS que evita el bloqueo del renderizado mientras carga la fuente; muestra primero una fuente del sistema
Skip link	Enlace oculto visualmente que salta al contenido principal; permite a usuarios de teclado evitar la navegación repetitiva
Animación compuesta (GPU)	Animación que solo usa <code>transform</code> u <code>opacity</code> ; el navegador la delega al compositor de la GPU sin repaints en CPU
WebP	Formato de imagen moderno (Google, 2010) con 30-50% menos peso que JPEG/PNG; soportado en todos los navegadores modernos
preconnect	Directiva HTML que establece DNS+TCP+TLS preventivamente con un dominio para acelerar futuros recursos
dns-prefetch	Directiva HTML más ligera que preconnect: solo resuelve el DNS del dominio, sin establecer conexión
FOIT	Flash of Invisible Text: texto invisible mientras carga la fuente; se evita con <code>font-display: swap</code>